

Submission Worksheet

IT202-450-M2026 / it202-milestone-1-2026

Submission Data

Course	IT202-450-M2026
Assignment	IT202 Milestone 1 - 2026
Student	Shakir A. (sa3327)
Status	submitted
Worksheet Progress	100%
Potential Grade	NaN/NaN (NaN%)
Updated	7/13/2026, 8:29:42 PM
Grading Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-1-2026/grading/sa3327
View Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-1-2026/sa3327

Instructions

Verify that your lesson-built Milestone 1 account system is complete and functional, then collect focused evidence for the account, access, role, profile, and styling requirements. Most of the coding should already be done from the lessons; this worksheet focuses on confirming the required behavior, polishing the project CSS, and explaining your understanding.

Assignment Setup

- Required branch: `Milestone1`
- Required starting branch: `qa`
- Required project folder: `public_html/project/`
- Required deployed evidence target: working `qa` site before production promotion
- Recommended order:
 - Read the whole worksheet.
 - Review the Milestone 1 assignment requirements.
 - Check out `qa`, pull the latest version, and create the milestone branch.
 - Ensure the related lessons are complete in your project.
 - Test each feature and requirement.
 - Apply your custom CSS to replace the temporary lesson example CSS.
 - Merge the milestone branch into `qa`, test deployed behavior, then collect worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the milestone branch with `git checkout -b Milestone1`.
4. Commit focused verification, fixes, and styling updates in small pieces. If there's nothing to commit, wait until you add the PDF to your repository.
5. Push `Milestone1`.
6. Open a pull request from `Milestone1` into `qa` and keep it open while you work.

you add the PDF to your repository.

5. Push `Milestone1`.
6. Open a pull request from `Milestone1` into `qa` and keep it open while you work.
7. After all Milestone 1 requirements pass locally, merge the pull request into `qa`.
8. Gather evidence from the deployed `qa` site and fill out the worksheet.
9. Export the worksheet PDF from the Learn Courses Platform.
10. Save the PDF in the repository under `docs/milestone01/`.
11. Add, commit, and push the PDF to the `Milestone1` branch.
 - `git add .`
 - `git commit -m "Add milestone 1 worksheet PDF"`
 - `git push origin Milestone1`
12. Upload the same PDF to Canvas.
13. Create and merge a pull request from `qa` into `prod` when production evidence is required.
14. Switch back to `qa`.
15. Pull the latest `qa` branch.

Shared Requirements

Apply these requirements to every Milestone 1 feature:

- Add a comment with your UCID, date, and a brief summary of the code block or feature area.
- Follow the shared helper, validation, flash message, session, and role-check patterns from the lessons.
- Replace the temporary lesson CSS with your own project styling.
- Use user-friendly flash messages for success and failure states.
- Log technical errors with `error_log()` instead of showing raw technical output to users.
- Use HTML validation attributes, JavaScript validation through the page's `validate()` function, and PHP validation before database writes.
- Keep safe form values sticky after validation errors.
- Clear password fields after validation errors.
- Test pages through the supported deployed `qa` URLs before collecting browser evidence.
- Use your own code, database behavior, branch, pull request, deployed URLs, and explanation evidence.

Evidence Format

Most sections use the same evidence pattern:

- Images:
 - Show relevant code and matching browser output.
 - Make sure UCID/date comments are visible in code evidence.
 - Make sure the deployed `qa` URL is visible in browser output evidence.
 - One combined screenshot is acceptable when the code and output are both readable.
 - Use multiple screenshots when one image would be hard to read.
- URLs:
 - Provide direct GitHub file links from the milestone branch.
 - Provide anticipated production URLs for deployed PHP pages.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - PHP page URLs must end in `.php`; zero credit for that URL entry if it does not.
- Response:
 - Explain the logic in your own words.
 - Describe the steps or decisions your PHP, SQL, or JavaScript makes.
 - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the Milestone 1 branch has been pushed.
- Confirm the pull request into `qa` exists

Subsequent Effect

- Confirm the Milestone 1 branch has been pushed.
- Confirm the pull request into `qa` exists.
- Confirm meaningful Milestone 1 commits are visible.
- Confirm the deployed `qa` version was reviewed before promotion.
- Confirm registration, login, logout, protected access, role access, profile edit, and password change were tested.
- Confirm the temporary lesson CSS has been replaced with your own styling.
- Confirm anticipated production URLs use the same path as the tested `qa` URLs with `-qa` changed to `-prod`.
- Confirm the worksheet PDF is committed under `docs/milestone01/` on the milestone branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress

100%

Users

Roles

UserRoles

100%

Details

- Show the **Users** table from the VS Code database extension so the structure and some data are visible.
- Show the **Roles** table from the VS Code database extension so the structure and some data are visible.
- Show the **UserRoles** table from the VS Code database extension so the structure and some data are visible.

[illegible]

Users

The screenshot shows a database interface with the 'Roles' table selected. The SQL query is `SELECT * FROM `Roles` LIMIT 100`. The table structure is as follows:

Name	Description	Is Active	Created	Modified
Admin	Can access project admin	1	2028-07-13 14:01:5	2028-07-13 15:06:4

Roles

id name age sex number married income education

0	John	20	M	1	no	10000	1
1	Mary	22	F	2	yes	12000	2
2	David	25	M	3	no	15000	3
3	Jane	28	F	4	yes	18000	4
4	Michael	30	M	5	no	20000	5
5	Emily	32	F	6	yes	22000	6
6	Robert	35	M	7	no	25000	7
7	Laura	38	F	8	yes	28000	8
8	James	40	M	9	no	30000	9
9	Sarah	42	F	10	yes	32000	10

UserRoles



100%

100%

Details

- Paste the direct GitHub file link for `sql/001_create_users.sql`.
- Paste the direct GitHub file link for `sql/003_create_table_roles.sql`.
- Paste the direct GitHub file link for `sql/004_create_table_userroles.sql`.

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/sql/001_create_users.sql 👍

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/sql/003_create_table_roles.sql 👍

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/sql/004_create_table_userroles.sql 👍

100%

Details

- Explain how the three tables work together.
- Explain where username and email uniqueness are enforced.

Answer this

The three tables (users, roles, and userroles) are used to implement a role-based access control (RBAC) system. The users table stores user information, the roles table stores role information, and the userroles table links users to their respective roles. The system enforces uniqueness of usernames and emails in the users table, and ensures that each user is associated with at least one role through the userroles table.

Submit your answer

Submit

Section #2: (1 pt.) Shared Setup And Helpers

100%

- Goal: show that the project loads and uses reusable account helpers from a shared setup file.
- Expected helper behavior:
 - Shared app setup loads the database, session, validation helpers, user helpers, role helpers, flash helpers, and other project utilities.
 - User helper functions safely check login state before reading user session values.
 - Role helper functions can check the current user's roles.

Task #1 (1 pt.) – Shared Helper Evidence

100%

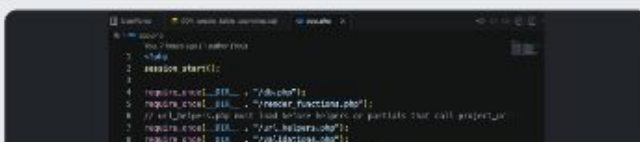
Combo #1

Part 1: Images

100%

Details

- Show `lib/app.php` with the shared helper imports visible.
- The screenshot should show references to the database, session, validation, flash, user, role, render, duplicate-user-detail, and URL helper files.



```

1 <?php
2 session_start();
3
4 require_once __DIR__ . "/helpers.php";
5 require_once __DIR__ . "/router_functions.php";
6 // url_helpers.php must load before helpers or partially load will experience
7 require_once __DIR__ . "/url_helpers.php";
8 require_once __DIR__ . "/validation.php";
9 // Keep user/helpers.php before role_helpers.php
10 // Has role_helpers depends on it, loaded last.
11 require_once __DIR__ . "/role_helpers.php";
12 require_once __DIR__ . "/flash_messages.php";
13 require_once __DIR__ . "/duplicate_user_email.php";
14 // register_route() depends on flash and validation.
15 require_once __DIR__ . "/role_helpers.php";
16 // ...

```

app.php

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `lib/app.php`.

URL #1

<https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/lib/app.php> 🟢

Part 3: Response 100%

Details

- Explain how `lib/app.php` loads the shared helper files.
- Explain why the specific load order matters.

Response

All of the shared helper files have been included by requireonce in the lib/app.php. By using requireonce it avoids files being included more than once and they can be used across the application. Load order is important because other helper files depend on functions from earlier files. Rolehelpers.php for instance relies on functions in userhelpers.php, flashmessages.php and urlhelpers.php.

Supports **bold**, *italic*, [links](url), `code`, etc.

394 chars

Section #3: (2 pts.) Registration

100%

- Goal: show that users can register with username, email, password, and confirm password.
- Expected behavior:
 - HTML, JavaScript, and PHP validation are all present.
 - Duplicate username and duplicate email cases show specific, user-friendly feedback.
 - Passwords are stored as hashes.
 - Safe form values remain sticky after validation errors.

Task #1 (2 pts.) – Registration Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the registration form code, including username, email, password, confirm password, and the escaped username pattern.
- Show the JavaScript `validate()` logic for registration.
- Show the PHP validation and insert logic, including password hashing and duplicate handling.
- Show deployed qa browser evidence for successful registration, duplicate username, duplicate email, and one validation failure.

- Explain how duplicate username and email feedback is shown.
- Explain how the password is stored safely.

100%

100%

 Part 1: Images **100%**

Details

- Show the login form code with the single username-or-email identifier field.
- Show the JavaScript `validate()` logic for login.
- Show the PHP login lookup, password verification, and session assignment logic.
- Show deployed `qa` browser evidence for login with email, login with username, invalid credentials, logout, and protected access after logout.
- For the login evidence, capturing the network request in developer tools is a good way to show the differences

```

100 # Create a new instance of the class
101 obj = MyClass()
102
103 # Call the method
104 obj.myMethod(10)
105
106 # Print the result
107 print(obj.myAttribute)
```

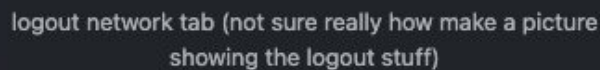
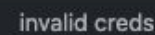
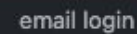
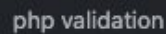
login form

```

187:         return result
188:     }
189:     // If the user is not logged in, we need to redirect them to the login page
190:     if (!user) {
191:         return res.redirect('/login');
192:     }
193:     // If the user is logged in, we need to check if they are an admin
194:     if (user.role === 'admin') {
195:         // Admins can access all routes
196:         return res.render('admin/index');
197:     }
198:     // If the user is not an admin, we need to redirect them to the user dashboard
199:     return res.render('user/index');
200: }
201:
202: // GET /api/users/:id
203: router.get('/api/users/:id', async (req, res) => {
204:     const { id } = req.params;
205:     const user = await User.findById(id);
206:     if (!user) {
207:         return res.status(404).json({ message: 'User not found' });
208:     }
209:     // If the user is an admin, we need to return all users
210:     if (req.user.role === 'admin') {
211:         return res.json(user);
212:     }
213:     // If the user is not an admin, we need to return only their own profile
214:     return res.json({ name: user.name, email: user.email });
215: });
216:
217: // GET /api/users
218: router.get('/api/users', async (req, res) => {
219:     const users = await User.find();
220:     return res.json(users);
221: });
222:
223: // POST /api/users
224: router.post('/api/users', async (req, res) => {
225:     const { name, email, password } = req.body;
226:     const user = await User.create({ name, email, password });
227:     return res.json(user);
228: });
229:
230: // PUT /api/users/:id
231: router.put('/api/users/:id', async (req, res) => {
232:     const { id } = req.params;
233:     const { name, email, password } = req.body;
234:     const user = await User.findByIdAndUpdate(id, { name, email, password });
235:     return res.json(user);
236: });
237:
238: // DELETE /api/users/:id
239: router.delete('/api/users/:id', async (req, res) => {
240:     const { id } = req.params;
241:     const user = await User.findByIdAndDelete(id);
242:     if (!user) {
243:         return res.status(404).json({ message: 'User not found' });
244:     }
245:     return res.json({ message: 'User deleted' });
246: });
247:
248: // GET /api/products
249: router.get('/api/products', async (req, res) => {
250:     const products = await Product.find();
251:     return res.json(products);
252: });
253:
254: // POST /api/products
255: router.post('/api/products', async (req, res) => {
256:     const { name, price, description } = req.body;
257:     const product = await Product.create({ name, price, description });
258:     return res.json(product);
259: });
260:
261: // PUT /api/products/:id
262: router.put('/api/products/:id', async (req, res) => {
263:     const { id } = req.params;
264:     const { name, price, description } = req.body;
265:     const product = await Product.findByIdAndUpdate(id, { name, price, description });
266:     return res.json(product);
267: });
268:
269: // DELETE /api/products/:id
270: router.delete('/api/products/:id', async (req, res) => {
271:     const { id } = req.params;
272:     const product = await Product.findByIdAndDelete(id);
273:     if (!product) {
274:         return res.status(404).json({ message: 'Product not found' });
275:     }
276:     return res.json({ message: 'Product deleted' });
277: });
278:
279: // GET /api/orders
280: router.get('/api/orders', async (req, res) => {
281:     const orders = await Order.find();
282:     return res.json(orders);
283: });
284:
285: // POST /api/orders
286: router.post('/api/orders', async (req, res) => {
287:     const { user, products } = req.body;
288:     const order = await Order.create({ user, products });
289:     return res.json(order);
290: });
291:
292: // PUT /api/orders/:id
293: router.put('/api/orders/:id', async (req, res) => {
294:     const { id } = req.params;
295:     const { user, products } = req.body;
296:     const order = await Order.findByIdAndUpdate(id, { user, products });
297:     return res.json(order);
298: });
299:
300: // DELETE /api/orders/:id
301: router.delete('/api/orders/:id', async (req, res) => {
302:     const { id } = req.params;
303:     const order = await Order.findByIdAndDelete(id);
304:     if (!order) {
305:         return res.status(404).json({ message: 'Order not found' });
306:     }
307:     return res.json({ message: 'Order deleted' });
308: });
309:
310: // GET /api/reports
311: router.get('/api/reports', async (req, res) => {
312:     const reports = await Report.find();
313:     return res.json(reports);
314: });
315:
316: // POST /api/reports
317: router.post('/api/reports', async (req, res) => {
318:     const { title, content } = req.body;
319:     const report = await Report.create({ title, content });
320:     return res.json(report);
321: });
322:
323: // PUT /api/reports/:id
324: router.put('/api/reports/:id', async (req, res) => {
325:     const { id } = req.params;
326:     const { title, content } = req.body;
327:     const report = await Report.findByIdAndUpdate(id, { title, content });
328:     return res.json(report);
329: });
330:
331: // DELETE /api/reports/:id
332: router.delete('/api/reports/:id', async (req, res) => {
333:     const { id } = req.params;
334:     const report = await Report.findByIdAndDelete(id);
335:     if (!report) {
336:         return res.status(404).json({ message: 'Report not found' });
337:     }
338:     return res.json({ message: 'Report deleted' });
339: });
340:
341: // GET /api/settings
342: router.get('/api/settings', async (req, res) => {
343:     const settings = await Setting.find();
344:     return res.json(settings);
345: });
346:
347: // POST /api/settings
348: router.post('/api/settings', async (req, res) => {
349:     const { key, value } = req.body;
350:     const setting = await Setting.create({ key, value });
351:     return res.json(setting);
352: });
353:
354: // PUT /api/settings/:id
355: router.put('/api/settings/:id', async (req, res) => {
356:     const { id } = req.params;
357:     const { key, value } = req.body;
358:     const setting = await Setting.findByIdAndUpdate(id, { key, value });
359:     return res.json(setting);
360: });
361:
362: // DELETE /api/settings/:id
363: router.delete('/api/settings/:id', async (req, res) => {
364:     const { id } = req.params;
365:     const setting = await Setting.findByIdAndDelete(id);
366:     if (!setting) {
367:         return res.status(404).json({ message: 'Setting not found' });
368:     }
369:     return res.json({ message: 'Setting deleted' });
370: });
371:
372: // GET /api/notifications
373: router.get('/api/notifications', async (req, res) => {
374:     const notifications = await Notification.find();
375:     return res.json(notifications);
376: });
377:
378: // POST /api/notifications
379: router.post('/api/notifications', async (req, res) => {
380:     const { title, message } = req.body;
381:     const notification = await Notification.create({ title, message });
382:     return res.json(notification);
383: });
384:
385: // PUT /api/notifications/:id
386: router.put('/api/notifications/:id', async (req, res) => {
387:     const { id } = req.params;
388:     const { title, message } = req.body;
389:     const notification = await Notification.findByIdAndUpdate(id, { title, message });
390:     return res.json(notification);
391: });
392:
393: // DELETE /api/notifications/:id
394: router.delete('/api/notifications/:id', async (req, res) => {
395:     const { id } = req.params;
396:     const notification = await Notification.findByIdAndDelete(id);
397:     if (!notification) {
398:         return res.status(404).json({ message: 'Notification not found' });
399:     }
400:     return res.json({ message: 'Notification deleted' });
401: });
402:
403: // GET /api/auth/logout
404: router.get('/api/auth/logout', async (req, res) => {
405:     const user = req.user;
406:     const token = req.token;
407:     const refreshToken = req.refreshToken;
408:     const { name, email } = user;
409:     const { message } = { message: 'Logout successful' };
410:     return res.json({ name, email, message });
411: });
412:
413: // GET /api/auth/login
414: router.get('/api/auth/login', async (req, res) => {
415:     const { email, password } = req.query;
416:     const user = await User.findOne({ email });
417:     if (!user) {
418:         return res.status(401).json({ message: 'Invalid email or password' });
419:     }
420:     const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1h' });
421:     const refreshToken = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_REFRESH_SECRET, { expiresIn: '7d' });
422:     return res.json({ token, refreshToken, user });
423: });
424:
425: // POST /api/auth/register
426: router.post('/api/auth/register', async (req, res) => {
427:     const { name, email, password } = req.body;
428:     const user = await User.create({ name, email, password });
429:     const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1h' });
430:     const refreshToken = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_REFRESH_SECRET, { expiresIn: '7d' });
431:     return res.json({ token, refreshToken, user });
432: });
433:
434: // PUT /api/auth/reset-password
435: router.put('/api/auth/reset-password', async (req, res) => {
436:     const { email, password } = req.body;
437:     const user = await User.findOne({ email });
438:     if (!user) {
439:         return res.status(401).json({ message: 'Invalid email or password' });
440:     }
441:     const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1h' });
442:     const refreshToken = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_REFRESH_SECRET, { expiresIn: '7d' });
443:     return res.json({ token, refreshToken, user });
444: });
445:
446: // GET /api/auth/verify
447: router.get('/api/auth/verify', async (req, res) => {
448:     const { token } = req.query;
449:     const user = await User.findOne({ token });
450:     if (!user) {
451:         return res.status(401).json({ message: 'Invalid token' });
452:     }
453:     return res.json({ user });
454: });
455:
456: // GET /api/auth/refresh
457: router.get('/api/auth/refresh', async (req, res) => {
458:     const { refreshToken } = req.query;
459:     const user = await User.findOne({ refreshToken });
460:     if (!user) {
461:         return res.status(401).json({ message: 'Invalid refresh token' });
462:     }
463:     const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1h' });
464:     const refreshToken = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_REFRESH_SECRET, { expiresIn: '7d' });
465:     return res.json({ token, refreshToken, user });
466: });
467:
468: // GET /api/auth/logout
469: router.get('/api/auth/logout', async (req, res) => {
470:     const user = req.user;
471:     const token = req.token;
472:     const refreshToken = req.refreshToken;
473:     const { name, email } = user;
474:     const { message } = { message: 'Logout successful' };
475:     return res.json({ name, email, message });
476: });
477:
478: // GET /api/auth/login
479: router.get('/api/auth/login', async (req, res) => {
480:     const { email, password } = req.query;
481:     const user = await User.findOne({ email });
482:     if (!user) {
483:         return res.status(401).json({ message: 'Invalid email or password' });
484:     }
485:     const token = jwt.sign
```

validate

validate



100%

Details

- Paste the direct GitHub file link for `public_html/project/login.php`.
- Paste the direct GitHub file link for `public_html/project/logout.php`.
- Paste anticipated production URLs for the login and logout pages.

100%

Details

- Explain how the identifier can match either username or email.
- Explain how the password check works.
- Explain how logout clears access.

The login check validates input to the username and the email fields, thus user will have access via email or username. Passwordverify() compares input password with password in database and login will succeed if it match. Logout with sessionunset() and session_destroy() and redirect user to login form page.

Supports ****bold****, **italic**, [links](url), `code`, etc.

309 chars

Section #5: (2 pts.) Profile

100%

- Goal: show that users can view and update profile information safely.
- Expected account-details behavior:
 - Profile view shows safe account information.
 - Username and email can be updated without the current password.
 - Duplicate username and email cases show specific, user-friendly feedback.
 - HTML, JavaScript, and PHP validation are all present.
- Expected password-change behavior:
 - Current password is required before saving a new password.
 - New password and confirm password are validated.
 - Successful password changes store a new hash,

Task #1 (2 pts.) – Profile Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the profile view and edit form code.
- Show the account-details update logic.
- Show the password-change logic.
- Show deployed **qa** browser evidence for profile view, account-details update, duplicate profile feedback, password-change failure with wrong current password, and successful password change.

```
1 <!-- Profile View and Edit Form -->
2 <?php
3     // Get user ID from session
4     $user_id = $_SESSION['user_id'];
5
6     // Fetch user details from database
7     $stmt = $db->prepare("SELECT * FROM users WHERE id = ?");
8     $stmt->execute($user_id);
9     $user = $stmt->fetch(PDO::FETCH_ASSOC);
10
11     // Check if user is logged in
12     if (!$user) {
13         header("Location: login.php");
14         exit;
15     }
16
17     // Form action
18     $update_url = "update_profile.php?id=" . $user_id;
19
20     // Form fields
21     $username = $user['username'];
22     $email = $user['email'];
23     $password = $user['password'];
24     $confirm_password = $password;
25
26     // Form validation
27     if (isset($_POST['update'])) {
28         $username = $_POST['username'];
29         $email = $_POST['email'];
30         $password = $_POST['password'];
31         $confirm_password = $_POST['confirm_password'];
32
33         // Validate form
34         if (empty($username) || empty($email) || empty($password) || empty($confirm_password)) {
35             $error = "All fields are required.";
36         } else if ($password != $confirm_password) {
37             $error = "Passwords do not match.";
38         } else {
39             // Update user details
40             $stmt = $db->prepare("UPDATE users SET username = ?, email = ?, password = ? WHERE id = ?");
41             $stmt->execute($username, $email, $password, $user_id);
42
43             // Success message
44             $success = "Profile updated successfully.";
45         }
46     }
47
48     // Display form
49     <div>
50         <h3>Profile</h3>
51         <div>
52             <div>
53                 <input type="text" value="{$username}" />
54                 <input type="text" value="{$email}" />
55                 <input type="password" value="{$password}" />
56                 <input type="password" value="{$confirm_password}" />
57             </div>
58             <input type="button" value="Update Profile" />
59         </div>
60     </div>
```

form code

```
1 <!-- Account Details Update Logic -->
2 <?php
3     // Get user ID from session
4     $user_id = $_SESSION['user_id'];
5
6     // Fetch user details from database
7     $stmt = $db->prepare("SELECT * FROM users WHERE id = ?");
8     $stmt->execute($user_id);
9     $user = $stmt->fetch(PDO::FETCH_ASSOC);
10
11     // Check if user is logged in
12     if (!$user) {
13         header("Location: login.php");
14         exit;
15     }
16
17     // Form action
18     $update_url = "update_profile.php?id=" . $user_id;
19
20     // Form fields
21     $username = $user['username'];
22     $email = $user['email'];
23     $password = $user['password'];
24     $confirm_password = $password;
25
26     // Form validation
27     if (isset($_POST['update'])) {
28         $username = $_POST['username'];
29         $email = $_POST['email'];
30         $password = $_POST['password'];
31         $confirm_password = $_POST['confirm_password'];
32
33         // Validate form
34         if (empty($username) || empty($email) || empty($password) || empty($confirm_password)) {
35             $error = "All fields are required.";
36         } else if ($password != $confirm_password) {
37             $error = "Passwords do not match.";
38         } else {
39             // Update user details
40             $stmt = $db->prepare("UPDATE users SET username = ?, email = ?, password = ? WHERE id = ?");
41             $stmt->execute($username, $email, $password, $user_id);
42
43             // Success message
44             $success = "Profile updated successfully.";
45         }
46     }
47
48     // Display form
49     <div>
50         <h3>Profile</h3>
51         <div>
52             <div>
53                 <input type="text" value="{$username}" />
54                 <input type="text" value="{$email}" />
55                 <input type="password" value="{$password}" />
56                 <input type="password" value="{$confirm_password}" />
57             </div>
58             <input type="button" value="Update Profile" />
59         </div>
60     </div>
```

acc update code

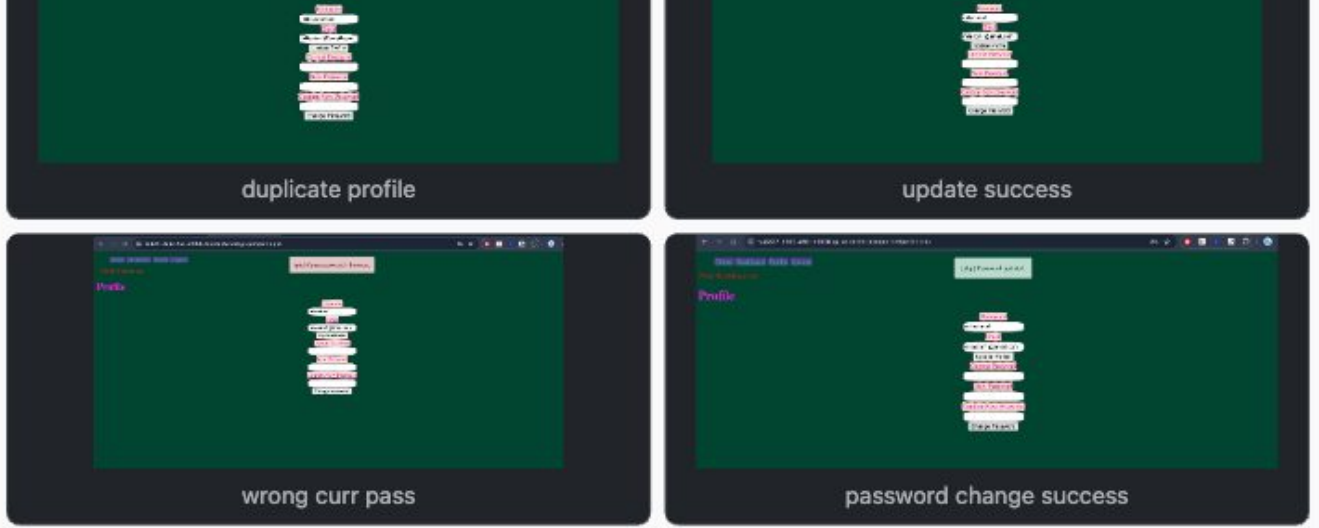
```
1 <!-- Password Change Logic -->
2 <?php
3     // Get user ID from session
4     $user_id = $_SESSION['user_id'];
5
6     // Fetch user details from database
7     $stmt = $db->prepare("SELECT * FROM users WHERE id = ?");
8     $stmt->execute($user_id);
9     $user = $stmt->fetch(PDO::FETCH_ASSOC);
10
11     // Check if user is logged in
12     if (!$user) {
13         header("Location: login.php");
14         exit;
15     }
16
17     // Form action
18     $update_url = "update_password.php?id=" . $user_id;
19
20     // Form fields
21     $current_password = $user['password'];
22     $new_password = $current_password;
23     $confirm_password = $current_password;
24
25     // Form validation
26     if (isset($_POST['change'])) {
27         $current_password = $_POST['current_password'];
28         $new_password = $_POST['new_password'];
29         $confirm_password = $_POST['confirm_password'];
30
31         // Validate form
32         if (empty($current_password) || empty($new_password) || empty($confirm_password)) {
33             $error = "All fields are required.";
34         } else if ($new_password != $confirm_password) {
35             $error = "New passwords do not match.";
36         } else if ($current_password != $user['password']) {
37             $error = "Current password is incorrect.";
38         } else {
39             // Update password
40             $stmt = $db->prepare("UPDATE users SET password = ? WHERE id = ?");
41             $stmt->execute($new_password, $user_id);
42
43             // Success message
44             $success = "Password changed successfully.";
45         }
46     }
47
48     // Display form
49     <div>
50         <h3>Change Password</h3>
51         <div>
52             <div>
53                 <input type="password" value="{$current_password}" />
54                 <input type="password" value="{$new_password}" />
55                 <input type="password" value="{$confirm_password}" />
56             </div>
57             <input type="button" value="Change Password" />
58         </div>
59     </div>
```

password change logic



profile view





Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `public_html/project/profile.php`.
- Paste the anticipated production URL for the profile page.

URL #1

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/profile.php 🟢

URL #2

<https://sa3327-it202-450-m2026-prod.onrender.com/project/profile.php> 🟢

Part 3: Response 100%

Details

- Explain the flow/process for changing username/email.
- Explain why password changes do require the current password.
- Explain how updated profile values persist after refresh.

Response

If email or username is updated, the input is validated then the User table updates, the new values will also be saved in the session.

Password change requires the current password to confirm ownership of the account and it uses `password_verify()` to match the hash.

The updated profile values persist because they are saved in the DB and loaded again.

Supports **bold**, *italic*, [links](url), `code`, etc.

354 chars

Section #6: (1 pt.) Access Roles And Navigation

100%

- Goal: show that access rules are enforced by the server, reflected in navigation, and presented with your own project styling.
- Expected behavior:
 - Logged-out users cannot access protected pages.
 - Users without the required role cannot access role-protected pages.
 - Navigation changes based on login state.

- Logged-out users cannot access protected pages.
- Users without the required role cannot access role-protected pages.
- Navigation changes based on login state.
- The project uses customized CSS instead of the temporary lesson example styling.
- Role pages from the lessons, such as create/list/assign roles, can satisfy the role-protected workflow.

Task #1 (0.67 pts.) – Access And Role Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the protected-page guard code.
- Show the role-protected guard code.
- Show the navigation code that changes based on login state.
- Show deployed qa browser evidence for a logged-out protected-page block, a wrong-role block, and a navigation state change.

```

1 <?php
2 require_once(__DIR__ . '/../../lib/app.php');
3 require_role("Admin");
4

```

role protected

```

12 <?php if (has_role("Admin")): ?>
13     <li><a href="/admin/create_role.php">?>Create Role</a>
14     <li><a href="/admin/list_roles.php">?>List Roles</a>
15     <li><a href="/admin/assign_roles.php">?>Assign Roles</a>
16 <?php endif; ?>

```

admin nav



Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `public_html/project/dashboard.php`.
- Paste the direct GitHub file link for `partials/nav.php`.
- Paste the direct GitHub file link for one role-protected admin page you tested.
- Paste anticipated production URLs for the protected and role-protected pages.

URL #1

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/dashboard.php



URL #2

<https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/partials/nav.php>



URL #3

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/admin/assign_roles.php



URL #4

https://sa3327-it202-450-m2026-prod.onrender.com/project/admin/assign_roles.php



URL #5

https://sa3327-it202-450-m2026-prod.onrender.com/project/admin/create_role.php



URL #6

URL #5
https://sa3327-it202-450-m2026-prod.onrender.com/project/admin/create_role.php 🟢

URL #6
https://sa3327-it202-450-m2026-prod.onrender.com/project/admin/list_roles.php 🟢

Part 3: Response 100%

Details

- Explain how the protected page checks login state.
- Explain how the role-protected page checks role access.
- Explain how navigation decides what to show.

Response

The protected dashboard uses `isLoggedIn()` to see if the user has logged in, and will redirect them to the login page if they have not. The admin page uses `requireRole("Admin")` so only users with the role "Admin" will have access to it. The navigation menu uses `isLoggedIn()` to display login or account links, and `hasRole("Admin")` to display the links to the admin pages to users with the "Admin" role only.

Supports **bold**, *italic*, [links](url), `code`, etc.

406 chars

Task #2 (0.33 pts.) – Project Styling Evidence

100%

Combo #1

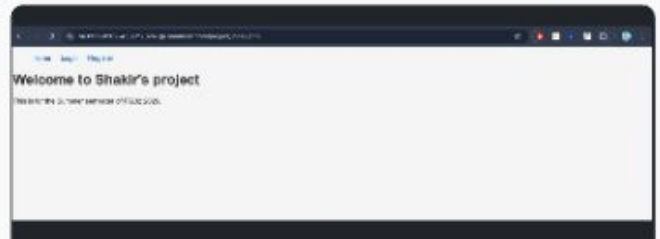
Part 1: Images 100%

Details

- Show the shared stylesheet or custom CSS changes.
- Show deployed **qa** browser evidence that the project no longer uses the temporary lesson example styling.



new CSS



qa new CSS

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `public_html/project/styles.css`.
- Paste the anticipated production URL for one styled project page.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/styles.css 🟢

URL #2
<https://sa3327-it202-450-m2026-qa.onrender.com/project/profile.php> 🟢

Part 3: Response 100%

Details

- Explain what you changed in the CSS to make the project your own.

Response

I changed the colors to make it modern and changed the font to Arial. I also removed unnecessary animations.

Supports **bold**, *italic*, [links](url), ``code``, etc.

109 chars

Section #7: (1 pt.) Misc

67%

Task #1 (0.33 pts.) – GitHub Details

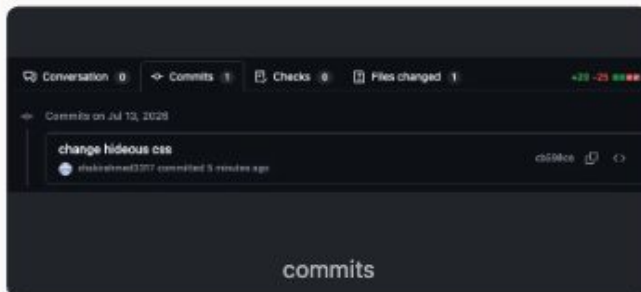
100%

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the pull request.
- The screenshot should show meaningful feature commits for Milestone 1.



Part 2: URLs 100%

Details

- Paste the pull request URL.
 - The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1

<https://github.com/shakirahmed3317/sa3327-it202-450-m2026/pull/51> 🍌

Task #2 (0.33 pts.) – WakaTime Activity

100%

Details

Details

- Visit the WakaTime dashboard.
- Open **Projects**.
- Find your repository.
- Screenshot the overall time near the repository name.
- Screenshot the file-level activity.
- The purpose is evidence of activity; the duration is not tied to the grade.
- The visual graphs are not required.



Task #3 (0.33 pts.) – Reflection

Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

I learned about client side & server side validation and to store secure data, session management, and to hash password for secure and to grant user roles accordingly.

Supports **bold**, *italic*, [links](url), 'code', etc.

167 chars

Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

The easiest part was understanding the HTML side of things, it was just basic forms and elements.

Supports **bold**, *italic*, [links](url), 'code', etc.

98 chars

 Saving...

Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

The hardest part was grasping the way the validation happened between HTML, JS, and PHP and making

Response

The hardest part was grasping the way the validation happened between HTML, JS, and PHP and making sure all three worked together. It also took some time to understand how the helper functions were doing and how they displayed the errors.

Supports **bold**, *italic*, [links](url), 'code', etc.

239 chars

End of Assignment - submission has been recorded.